Graph Utilities

Simon Frost

Overview

RDFLib.jl provides a collection of utility functions for analyzing, comparing, and transforming RDF graphs. This vignette covers graph statistics, isomorphism testing, set operations, concise bounded descriptions, and visualization.

```
using RDFLib

# Build a sample graph
g = RDFGraph()
ex = Namespace("http://example.org/")

add!(g, Triple(ex("alice"), RDF.type, FOAF("Person")))
add!(g, Triple(ex("alice"), FOAF("name"), Literal("Alice")))
add!(g, Triple(ex("alice"), FOAF("age"), Literal(30)))
add!(g, Triple(ex("alice"), FOAF("knows"), ex("bob")))
add!(g, Triple(ex("bob"), RDF.type, FOAF("Person")))
add!(g, Triple(ex("bob"), FOAF("name"), Literal("Bob")))
add!(g, Triple(ex("bob"), FOAF("age"), Literal(25)))
println("Graph has $(length(g)) triples")
```

```
Graph has 7 triples
```

Graph Statistics

The `graph_stats` function provides a summary of graph contents:

```
stats = graph_stats(g)
println("Triples:    $(stats.triples)")
println("Subjects:   $(stats.subjects)")
println("Predicates: $(stats.predicates)")
println("Objects:    $(stats.objects)")
println("URIRefs:    $(stats.uri_refs)")
println("BNodes:     $(stats.blank_nodes)")
println("Literals:   $(stats.literals)")
```

```
Triples:    7
Subjects:   2
Predicates: 4
Objects:    6
URIRefs:    7
BNodes:     0
Literals:   4
```

Graph Comparison

Set Operations

Graphs support standard set operations:

```
g1 = RDFGraph()
add!(g1, Triple(ex("a"), ex("p"), Literal("1")))
add!(g1, Triple(ex("a"), ex("p"), Literal("2")))
add!(g1, Triple(ex("b"), ex("p"), Literal("3")))

g2 = RDFGraph()
add!(g2, Triple(ex("a"), ex("p"), Literal("2")))
add!(g2, Triple(ex("c"), ex("p"), Literal("4")))

# Union
merged = g1 + g2
println("Union: $(length(merged)) triples")

# Intersection
common = intersect(g1, g2)
println("Intersection: $(length(common)) triples")

# Difference
```

```
diff = setdiff(g1, g2)
println("In g1 but not g2: $(length(diff)) triples")
```

```
Union: 4 triples
Intersection: 1 triples
In g1 but not g2: 2 triples
```

Graph Diff

`graph_diff` returns triples in both, only-in-first, and only-in-second:

```
both, only1, only2 = graph_diff(g1, g2)
println("In both:    $(length(both)) triples")
println("Only in g1: $(length(only1)) triples")
println("Only in g2: $(length(only2)) triples")
```

```
In both:    1 triples
Only in g1: 2 triples
Only in g2: 1 triples
```

Graph Isomorphism

Two graphs are isomorphic if they are identical up to blank node relabeling:

```
ga = RDFGraph()
add!(ga, Triple(ex("s"), ex("p"), BNode("x1")))
add!(ga, Triple(BNode("x1"), ex("q"), Literal("v")))

gb = RDFGraph()
add!(gb, Triple(ex("s"), ex("p"), BNode("z9")))
add!(gb, Triple(BNode("z9"), ex("q"), Literal("v")))

println("Isomorphic: ", isomorphic(ga, gb))
```

```
Isomorphic: true
```

Non-isomorphic graphs:

```
gc = RDFGraph()
add!(gc, Triple(ex("s"), ex("p"), BNode("a")))
add!(gc, Triple(BNode("a"), ex("q"), Literal("different")))

println("Isomorphic: ", isomorphic(ga, gc))
```

```
Isomorphic: false
```

Concise Bounded Description (CBD)

CBD extracts all triples about a resource, recursively following blank nodes:

```
g_cbd = RDFGraph()
b = BNode()
add!(g_cbd, Triple(ex("alice"), FOAF("name"), Literal("Alice")))
add!(g_cbd, Triple(ex("alice"), ex("address"), b))
add!(g_cbd, Triple(b, ex("street"), Literal("123 Main St")))
add!(g_cbd, Triple(b, ex("city"), Literal("Springfield")))
add!(g_cbd, Triple(ex("bob"), FOAF("name"), Literal("Bob")))

desc = cbd(g_cbd, ex("alice"))
println("CBD of alice: $(length(desc)) triples")
for t in triples(desc, (nothing, nothing, nothing))
    println("  $(t.subject) $(t.predicate) $(t.object)")
end
```

```
CBD of alice: 4 triples
  URIRef("http://example.org/alice") URIRef("http://xmlns.com/foaf/0.1/name") Literal("Alice"
  URIRef("http://example.org/alice") URIRef("http://example.org/address") BNode("N27d71f6802
  BNode("N27d71f68023db02bdaa6d10bee39d79b") URIRef("http://example.org/street") Literal("12
  BNode("N27d71f68023db02bdaa6d10bee39d79b") URIRef("http://example.org/city") Literal("Spri
```

Subjects, Predicates, Objects

Convenience accessors for graph components:

```
println("Subjects:")
for s in subjects(g, RDF.type, FOAF("Person"))
    println("  $s")
```

```
end

println("\nPredicates for alice:")
for p in predicates(g, ex("alice"), nothing)
    println("  $p")
end

println("\nObjects - ages:")
for o in objects(g, nothing, FOAF("age"))
    println("  $o")
end
```

```
Subjects:
  URIRef("http://example.org/alice")
  URIRef("http://example.org/bob")

Predicates for alice:
  URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type")
  URIRef("http://xmlns.com/foaf/0.1/name")
  URIRef("http://xmlns.com/foaf/0.1/age")
  URIRef("http://xmlns.com/foaf/0.1/knows")

Objects - ages:
  Literal("30", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
  Literal("25", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
```

Transitive Traversal

Follow a property transitively to find all reachable objects:

```
g_trans = RDFGraph()
add!(g_trans, Triple(ex("a"), ex("parent"), ex("b")))
add!(g_trans, Triple(ex("b"), ex("parent"), ex("c")))
add!(g_trans, Triple(ex("c"), ex("parent"), ex("d")))

reachable = transitive_objects(g_trans, ex("a"), ex("parent"))
println("Ancestors of a (transitive):")
for obj in reachable
    println("  $obj")
end
```

```
Ancestors of a (transitive):
  URIRef("http://example.org/c")
  URIRef("http://example.org/a")
  URIRef("http://example.org/b")
  URIRef("http://example.org/d")
```

You can also traverse backwards with `transitive_subjects`:

```
descendants = transitive_subjects(g_trans, ex("d"), ex("parent"))
println("Descendants of d (inverse transitive):")
for subj in descendants
    println("  $subj")
end
```

```
Descendants of d (inverse transitive):
  URIRef("http://example.org/c")
  URIRef("http://example.org/a")
  URIRef("http://example.org/b")
  URIRef("http://example.org/d")
```

Skolemization

Replace blank nodes with deterministic URIs:

```
g_bnode = RDFGraph()
b1 = BNode()
add!(g_bnode, Triple(ex("s"), ex("p"), b1))
add!(g_bnode, Triple(b1, ex("q"), Literal("hello")))

g_skolem = skolemize(g_bnode)
println("Skolemized graph:")
for t in triples(g_skolem, (nothing, nothing, nothing))
    println("  $(t.subject) $(t.predicate) $(t.object)")
end
```

```
Skolemized graph:
  URIRef("http://example.org/s") URIRef("http://example.org/p") URIRef("https://rdflib.github
known/genid/Nc7f0051d8d4f600a6b1297c20503050d")
  URIRef("https://rdflib.github.io/.well-known/genid/Nc7f0051d8d4f600a6b1297c20503050d") URI
```

Visualization

Generate DOT format for Graphviz rendering:

```
g_viz = RDFGraph()
add!(g_viz, Triple(ex("alice"), FOAF("name"), Literal("Alice")))
add!(g_viz, Triple(ex("alice"), FOAF("knows"), ex("bob")))
add!(g_viz, Triple(ex("bob"), FOAF("name"), Literal("Bob")))

dot = to_dot(g_viz; label="Friendship Graph")
println(dot)
```

```
digraph {
  rankdir=LR;
  label="Friendship Graph";

  n1 [shape=box, label="Alice"];
  n2 [label="ns1:alice"];
  n3 [shape=box, label="Bob"];
  n4 [label="ns1:bob"];

  n2 -> n1 [label="ns2:name"];
  n2 -> n4 [label="ns2:knows"];
  n4 -> n3 [label="ns2:name"];
}
```

To render this as an image, pipe the output to Graphviz:

```
# Save to file
save_visualization(g_viz, "graph.png"; label="Friendship Graph")

# Or pipe to Graphviz manually
# echo "$dot" | dot -Tpng -o graph.png
```

VoID Descriptions

Generate VoID (Vocabulary of Interlinked Datasets) metadata:

```
void_graph = generate_void(g, ex("myDataset"); title="Example Dataset")
println("VoID graph: $(length(void_graph)) triples")
println(serialize(void_graph, TurtleFormat()))
```

```
VoID graph: 22 triples
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix ns1: <http://example.org/> .
@prefix ns2: <http://xmlns.com/foaf/0.1/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix void: <http://rdfs.org/ns/void#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:myDataset a void:Dataset ;
    dcterms:title "Example Dataset" ;
    void:classPartition [ void:class ns2:Person ;
        void:entities 2 ] ;
    void:classes 1 ;
    void:distinctObjects 6 ;
    void:distinctSubjects 2 ;
    void:properties 4 ;
    void:propertyPartition [ void:property ns2:age ;
        void:triples 2 ],
        [ void:property rdf:type ;
        void:triples 2 ],
        [ void:property ns2:name ;
        void:triples 2 ],
        [ void:property ns2:knows ;
        void:triples 1 ] ;
    void:triples 7 .
```

What's Next?

- **SPARQL Queries** — query graphs with the full SPARQL 1.1 language (vignette 06)
- **Property Paths** — navigate graphs with `/`, `*`, `+`, `^` (vignette 16)
- **SHACL Validation** — validate graph structure against shapes (vignette 11)